

## Data Science Laboratory – Exercise 13.11.25 (Data-Extraction-Preprocessing)

## 1. Reading the dataset from excel format

Code:

```
## 1. Reading the dataset from excel format
base_dir = os.path.dirname(os.path.abspath(__file__))
file_path = os.path.join(base_dir, '..', 'dataset', 'smartphones.csv')
df = pd.read_csv(file_path)

print("SmartPhone Specs Details Collection")
print(df.head(10))
```

Output:

```
SmartPhone Specs Details Collection
  brand_name      model  ...  resolution_height  resolution_width
0    apple  Apple iPhone 11  ...             1792             828
1    apple  Apple iPhone 11 (128GB)  ...             1792             828
2    apple  Apple iPhone 11 Pro Max  ...             2688            1242
3    apple  Apple iPhone 12  ...             2532            1170
4    apple  Apple iPhone 12 (128GB)  ...             2532            1170
5    apple  Apple iPhone 12 (256GB)  ...             2532            1170
6    apple  Apple iPhone 12 Mini  ...             2340            1080
7    apple  Apple iPhone 12 Mini (128GB)  ...             2340            1080
8    apple  Apple iPhone 12 Mini (256GB)  ...             2340            1080
9    apple  Apple iPhone 12 Pro (256GB)  ...             2532            1170
```

## 2. Display DataType of resolution\_width column

Code:

```
## 2. Display DataType of resolution_width column
print("Datatype of resolution_width column: ", df['resolution_width'].dtype)
```

Output:

```
Datatype of resolution_width column: int64
```

### 3. Renaming the column names

Code:

```
## 3. Renaming the column names
df.rename(columns = {'avg_rating' : 'Average_Rating', 'battery_capacity' : 'Battery_Capacity'}, inplace=True)
print("Columns Renamed:\n", df.columns)
```

Result:

```
Columns Renamed:
Index(['brand_name', 'model', 'price', 'Average_Rating', '5G_or_not',
      'processor_brand', 'num_cores', 'processor_speed', 'Battery_Capacity',
      'fast_charging_available', 'fast_charging', 'ram_capacity',
      'internal_memory', 'screen_size', 'refresh_rate', 'num_rear_cameras',
      'os', 'primary_camera_rear', 'primary_camera_front',
      'extended_memory_available', 'resolution_height', 'resolution_width'],
      dtype='object')
```

### 4. Add additional column

Code:

```
## 4. Add additional column
df['Available_On_Flipkart'] = df['5G_or_not']
print(df.head())
```

Result:

|   | brand_name | model                   | price  | ... | resolution_height | resolution_width | Available_On_Flipkart |
|---|------------|-------------------------|--------|-----|-------------------|------------------|-----------------------|
| 0 | apple      | Apple iPhone 11         | 38999  | ... | 1792              | 828              | 0                     |
| 1 | apple      | Apple iPhone 11 (128GB) | 46999  | ... | 1792              | 828              | 0                     |
| 2 | apple      | Apple iPhone 11 Pro Max | 109900 | ... | 2688              | 1242             | 0                     |
| 3 | apple      | Apple iPhone 12         | 51999  | ... | 2532              | 1170             | 1                     |
| 4 | apple      | Apple iPhone 12 (128GB) | 55999  | ... | 2532              | 1170             | 1                     |

### 5. Find correlation

Code:

```
## 5. Find Correlation
print("Correlation Matrix: ", df.corr(numeric_only=True))
```

Result:

```
Correlation Matrix:
price      1.000000    0.283504    0.305066    ...    0.353578    0.340592    0.305066
Average_Rating 0.283504    1.000000    0.596087    ...    0.688108    0.432582    0.596087
5G_or_not    0.305066    0.596087    1.000000    ...    0.504365    0.301828    1.000000
num_cores   -0.048561    0.199741    0.139607    ...    0.266491    0.137952    0.139607
processor_speed 0.474049    0.628446    0.609583    ...    0.553923    0.365757    0.609583
Battery_Capacity -0.159232   -0.015581   -0.013237    ...    0.036503   -0.024607   -0.013237
fast_charging_available 0.116739    0.542814    0.355858    ...    0.510112    0.318720    0.355858
fast_charging  0.277591    0.527613    0.440624    ...    0.331469    0.222381    0.440624
ram_capacity   0.386002    0.757613    0.533957    ...    0.578105    0.380274    0.533957
internal_memory 0.557168    0.481070    0.403837    ...    0.427218    0.341709    0.403837
screen_size   0.113253    0.298272    0.230598    ...    0.295926    0.334099    0.230598
refresh_rate   0.244115    0.610795    0.611794    ...    0.481937    0.294688    0.611794
num_rear_cameras 0.125330    0.515531    0.206512    ...    0.465547    0.342834    0.206512
primary_camera_rear 0.092095    0.562046    0.347918    ...    0.394871    0.257728    0.347918
primary_camera_front 0.163068    0.577973    0.358833    ...    0.416126    0.262344    0.358833
extended_memory_available -0.448628   -0.415265   -0.507752    ...   -0.353785   -0.249250   -0.507752
resolution_height 0.353578    0.688108    0.504365    ...    1.000000    0.265519    0.504365
resolution_width 0.340592    0.432582    0.301828    ...    0.265519    1.000000    0.301828
Available_On_Flipkart 0.305066    0.596087    1.000000    ...    0.504365    0.301828    1.000000

[19 rows x 19 columns]
```

## 6. Reading top 10 and bottom 10 records

Code:

```
## 6. Reading to 10 and bottom 10 records
print("Top 10 records: \n", df.head(10))
print("Bottom 10 records: \n", df.tail(10))
```

Result:

```
Top 10 records:
  brand_name      model  price  ...  resolution_height  resolution_width  Available_On_Flipkart
0    apple  Apple iPhone 11   38999  ...             1792             828              0
1    apple  Apple iPhone 11 (128GB)  46999  ...             1792             828              0
2    apple  Apple iPhone 11 Pro Max 109900  ...             2688            1242              0
3    apple  Apple iPhone 12   51999  ...             2532            1170              1
4    apple  Apple iPhone 12 (128GB)  55999  ...             2532            1170              1
5    apple  Apple iPhone 12 (256GB)  67999  ...             2532            1170              1
6    apple  Apple iPhone 12 Mini   40999  ...             2340            1080              1
7    apple  Apple iPhone 12 Mini (128GB)  45999  ...             2340            1080              1
8    apple  Apple iPhone 12 Mini (256GB)  55999  ...             2340            1080              1
9    apple  Apple iPhone 12 Pro (256GB) 119900  ...             2532            1170              1

[10 rows x 23 columns]
Bottom 10 records:
  brand_name      model  price  ...  resolution_height  resolution_width  Available_On_Flipkart
970  xiaomi  Xiaomi Redmi Note 13 Pro Max 5G  20499  ...             2460            1080              1
971  xiaomi  Xiaomi Redmi Note 4  10300  ...             1920            1080              0
972  xiaomi  Xiaomi Redmi Note 8 2021   9990  ...             2340            1080              0
973  xiaomi  Xiaomi Redmi Note 8 Pro  16999  ...             2340            1080              0
974  xiaomi  Xiaomi Redmi Note 9  11989  ...             2340            1080              0
975  xiaomi  Xiaomi Redmi Note 9 Pro  13999  ...             2400            1080              0
976  xiaomi  Xiaomi Redmi Note 9 Pro (4GB RAM + 128GB)  14439  ...             2400            1080              0
977  xiaomi  Xiaomi Redmi Note 9 Pro Max  16490  ...             2400            1080              0
978    zte  ZTE Axon 30S  19999  ...             2460            1080              1
979    zte  ZTE Axon 40 Ultra 5G  61990  ...             2480            1116              1

[10 rows x 23 columns]
```

## 7. Reading the header

Code:

```
## 7. Reading the header
print("Header Columns: ", df.columns.to_list())
```

Result

```
Header Columns: ['brand_name', 'model', 'price', 'Average_Rating', '5G_or_not', 'processor_brand', 'num_cores', 'processor_speed', 'Battery_Capacity',
'fast_charging_available', 'fast_charging', 'ram_capacity', 'internal_memory', 'screen_size', 'refresh_rate', 'num_rear_cameras', 'os', 'primary_camera_rear', 'primary_camera_front', 'extended_memory_available', 'resolution_height', 'resolution_width', 'Available_On_Flipkart']
```

## 8. Reading data on column basis

Code:

```
## 8. Reading data on column basis
print("Available_On_Flipkart Column Data: \n ", df['Available_On_Flipkart'].head(6))
```

Result:

```
Available_On_Flipkart Column Data:
  0  0
1  0
2  0
3  1
4  1
5  1
Name: Available_On_Flipkart, dtype: int64
```

## 9. Reading specific data

Code:

```
## 9. Reading Specific data
print("Specific Data [Row 2, Column 'Available_On_Flipkart']", df.loc[2, 'Available_On_Flipkart'])
```

Result:

```
Specific Data [Row 2, Column 'Available_On_Flipkart'] 0
```

10. Reading data of a specific tuple based on a) index ranges b) specific index

Code:

```
## 10. Reading data of a specific tuple
# (a) index ranges (b) specific index
print("Index Range 0-5: \n", df.iloc[0:5])
print("Specific Index (3): \n" , df.loc[3])
```

Result:

```
Specific Data [Row 2, Column 'Available_On_Flipkart'] 0
Index Range 0-5:
  brand_name      model  price  ...  resolution_height  resolution_width  Available_On_Flipkart
0   apple      Apple iPhone 11  38999  ...           1792             828                0
1   apple  Apple iPhone 11 (128GB)  46999  ...           1792             828                0
2   apple  Apple iPhone 11 Pro Max 109900  ...           2688            1242                0
3   apple      Apple iPhone 12  51999  ...           2532            1170                1
4   apple  Apple iPhone 12 (128GB)  55999  ...           2532            1170                1

[5 rows x 23 columns]
Specific Index (3):
  brand_name      apple
  model      Apple iPhone 12
  price      51999
  Average_Rating      7.4
  5G_or_not      1
  processor_brand      bionic
  num_cores      6.0
  processor_speed      3.1
  Battery_Capacity      NaN
  fast_charging_available      0
  fast_charging      NaN
  ram_capacity      4
  ram_capacity      4
  ram_capacity      4
  internal_memory      64
  screen_size      6.1
  refresh_rate      60
  num_rear_cameras      2
  os      ios
  primary_camera_rear      12.0
  primary_camera_front      12.0
  extended_memory_available      0
  resolution_height      2532
  resolution_width      1170
  Available_On_Flipkart      1
Name: 3, dtype: object
```

### 11. Rows with indexes 3 to 7

Code:

```
## 11. Rows with indexes 3 to 7
print(df.loc[3:7])
```

Output:

|   | brand_name | model                        | price | ... | resolution_height | resolution_width | Available_On_Flipkart |
|---|------------|------------------------------|-------|-----|-------------------|------------------|-----------------------|
| 3 | apple      | Apple iPhone 12              | 51999 | ... | 2532              | 1170             | 1                     |
| 4 | apple      | Apple iPhone 12 (128GB)      | 55999 | ... | 2532              | 1170             | 1                     |
| 5 | apple      | Apple iPhone 12 (256GB)      | 67999 | ... | 2532              | 1170             | 1                     |
| 6 | apple      | Apple iPhone 12 Mini         | 40999 | ... | 2340              | 1080             | 1                     |
| 7 | apple      | Apple iPhone 12 Mini (128GB) | 45999 | ... | 2340              | 1080             | 1                     |

[5 rows x 23 columns]

### 12. Display rows with indexes 5 to 0 (Reverse Order)

Code

```
## 12. Display rows with indexes 5 to 0 (Reverse Order)
print(df.loc[5::-1])
```

Output

|   | brand_name | model                   | price  | ... | resolution_height | resolution_width | Available_On_Flipkart |
|---|------------|-------------------------|--------|-----|-------------------|------------------|-----------------------|
| 5 | apple      | Apple iPhone 12 (256GB) | 67999  | ... | 2532              | 1170             | 1                     |
| 4 | apple      | Apple iPhone 12 (128GB) | 55999  | ... | 2532              | 1170             | 1                     |
| 3 | apple      | Apple iPhone 12         | 51999  | ... | 2532              | 1170             | 1                     |
| 2 | apple      | Apple iPhone 11 Pro Max | 109900 | ... | 2688              | 1242             | 0                     |
| 1 | apple      | Apple iPhone 11 (128GB) | 46999  | ... | 1792              | 828              | 0                     |
| 0 | apple      | Apple iPhone 11         | 38999  | ... | 1792              | 828              | 0                     |

[6 rows x 23 columns]

### 13. Display rows with odd indexes from 1 to 7

Code:

```
## 13 Display rows with odd indexes from 1 to 7
print(df.loc[1:7:2])
```

Output:

```
brand_name      model  price  ...  resolution_height  resolution_width  Available_On_Flipkart
1      apple  Apple iPhone 11 (128GB)  46999  ...      1792           828           0
3      apple      Apple iPhone 12  51999  ...      2532          1170           1
5      apple  Apple iPhone 12 (256GB)  67999  ...      2532          1170           1
7      apple  Apple iPhone 12 Mini (128GB)  45999  ...      2340          1080           1

[4 rows x 23 columns]
```

14. Display all the rows where “any condition”

Code:

```
## 14 Display all the rows where "any condition"
print(df[df['brand_name']=="jio"])
```

Output:

```
brand_name      model  price  ...  resolution_height  resolution_width  Available_On_Flipkart
180      jio      Jio JioPhone 5G  11990  ...      1600           720           1
181      jio      Jio JioPhone Next  4649  ...      1440           720           0
182      jio  Jio JioPhone Next (3GB RAM + 32GB)  7499  ...      1440           720           0
183      jio      Jio Phone 3  4499  ...      1280           720           0

[4 rows x 23 columns]
```

17. Display shape attribute

Code:

```
## 17 Display shape attribute
print("Shape of Dataset:", df.shape)
```

Output:

```
Shape of Dataset: (980, 23)
```

18. Filtering on multiple attributes

Code:

```
# 18 Filtering on multiple attributes (e.g., Apple brand AND ram > 4)
print("\n--- Filtering: Apple phones with more than 6GB RAM ---")
condition1 = df['brand_name'] == 'apple'
condition2 = df['ram_capacity'] > 6
print(df[condition1 & condition2])
```

Output:

```
--- Filtering: Apple phones with more than 6GB RAM ---
  brand_name      model  price  ...  resolution_height  resolution_width  Available_On_Fli
36   apple  Apple iPhone 15 Plus  84990  ...           2778           1284
37   apple  Apple iPhone 15 Pro 130990  ...           2532           1170
38   apple  Apple iPhone 15 Pro Max 142990  ...           2778           1284
39   apple  Apple iPhone 15 Ultra 149900  ...           2796           1290

[4 rows x 23 columns]
```

## 19 Filtering data on the basis of a particular pattern

Code:

```
# 19 Filtering data on the basis of a particular pattern (string contains)
print("\n--- Filtering: Models containing 'Pro' ---")
print(df[df['model'].str.contains('Pro', case=False, na=False)])
```

Output:

```
--- Filtering: Models containing 'Pro' ---
  brand_name      model  ...  resolution_width  Available_On_Flipkart
2   apple  Apple iPhone 11 Pro Max  ...           1242              0
9   apple  Apple iPhone 12 Pro (256GB)  ...           1170              1
10  apple  Apple iPhone 12 Pro (512GB)  ...           1170              1
15  apple  Apple iPhone 13 Pro  ...           1170              1
16  apple  Apple iPhone 13 Pro (1TB)  ...           1170              1
..   ...  ...  ...  ...  ...
970 xiaomi  Xiaomi Redmi Note 13 Pro Max 5G  ...           1080              1
973 xiaomi  Xiaomi Redmi Note 8 Pro  ...           1080              0
975 xiaomi  Xiaomi Redmi Note 9 Pro  ...           1080              0
976 xiaomi  Xiaomi Redmi Note 9 Pro (4GB RAM + 128GB)  ...           1080              0
977 xiaomi  Xiaomi Redmi Note 9 Pro Max  ...           1080              0

[203 rows x 23 columns]
```

## 20 Filtering data on basis of particular regex

Code

```
# 20 Filtering data on basis of particular regex (e.g., processor is 'bionic' OR 'tensor')
print("\n--- Filtering (Regex): Processor is 'bionic' or 'tensor' ---")
print(df[df['processor_brand'].str.contains(r'bionic|tensor', na=False)])
```



## Output

```
--- Filtering (Regex): Processor is 'bionic' or 'tensor' ---
  brand_name      model  ... resolution_width  Available_On_Flipkart
0    apple  Apple iPhone 11  ...           828              0
1    apple  Apple iPhone 11 (128GB)  ...           828              0
2    apple  Apple iPhone 11 Pro Max  ...          1242              0
3    apple  Apple iPhone 12  ...          1170              1
4    apple  Apple iPhone 12 (128GB)  ...          1170              1
5    apple  Apple iPhone 12 (256GB)  ...          1170              1
6    apple  Apple iPhone 12 Mini  ...          1080              1
7    apple  Apple iPhone 12 Mini (128GB)  ...          1080              1
8    apple  Apple iPhone 12 Mini (256GB)  ...          1080              1
9    apple  Apple iPhone 12 Pro (256GB)  ...          1170              1
```

## 21 Saving using location, attribute name, alter position

### Code

```
# 21 Saving using location, attribute name, alter position
df.insert(1, "InsertedNewColumn", df["processor_brand"])
print(df)
```

### Output:

```
[15 rows x 23 columns]
  brand_name  InsertedNewColumn  ... resolution_width  Available_On_Flipkart
0    apple          bionic  ...           828              0
1    apple          bionic  ...           828              0
2    apple          bionic  ...          1242              0
3    apple          bionic  ...          1170              1
4    apple          bionic  ...          1170              1
..      ...                ...  ...              ...              ...
975  xiaomi      snapdragon  ...          1080              0
976  xiaomi      snapdragon  ...          1080              0
977  xiaomi      snapdragon  ...          1080              0
978    zte      snapdragon  ...          1080              1
979    zte      snapdragon  ...          1116              1
```

```
[980 rows x 24 columns]
```

## 23. Dropping columns, range

### Code:

```
# 23 Dropping columns, range
df2 = df.drop(columns = ["InsertedNewColumn"])
df3 = df.drop(df.columns[2:4], axis = 1)
print(df2)
print(df3)
```

Output:

|     | brand_name | ... | Available_On_Flipkart |
|-----|------------|-----|-----------------------|
| 0   | apple      | ... | 0                     |
| 1   | apple      | ... | 0                     |
| 2   | apple      | ... | 0                     |
| 3   | apple      | ... | 1                     |
| 4   | apple      | ... | 1                     |
| ..  | ...        | ... | ...                   |
| 975 | xiaomi     | ... | 0                     |
| 976 | xiaomi     | ... | 0                     |
| 977 | xiaomi     | ... | 0                     |
| 978 | zte        | ... | 1                     |
| 979 | zte        | ... | 1                     |

[980 rows x 23 columns]

|     | brand_name | InsertedNewColumn | ... | resolution_width | Available_On_Flipkart |
|-----|------------|-------------------|-----|------------------|-----------------------|
| 0   | apple      | bionic            | ... | 828              | 0                     |
| 1   | apple      | bionic            | ... | 828              | 0                     |
| 2   | apple      | bionic            | ... | 1242             | 0                     |
| 3   | apple      | bionic            | ... | 1170             | 1                     |
| 4   | apple      | bionic            | ... | 1170             | 1                     |
| ..  | ...        | ...               | ... | ...              | ...                   |
| 975 | xiaomi     | snapdragon        | ... | 1080             | 0                     |
| 976 | xiaomi     | snapdragon        | ... | 1080             | 0                     |
| 977 | xiaomi     | snapdragon        | ... | 1080             | 0                     |
| 978 | zte        | snapdragon        | ... | 1080             | 1                     |
| 979 | zte        | snapdragon        | ... | 1116             | 1                     |

[980 rows x 22 columns]

## 24. Sorting (ascending, multiple attributes)

Code:

```
# 24 Sorting (ascending, multiple attributes)
print(df.sort_values(by=["Average_Rating"], ascending=[True]))
```

Output:

```
brand_name  InsertedNewColumn  ... resolution_width  Available_On_Flipkart
45    apple          bionic  ...           828           0
499   realme          tiger  ...           720           0
44    apple          bionic  ...           750           1
108   ikall           NaN  ...           720           0
907   xiaomi          helio  ...           720           0
..      ...              ...  ...           ...           ...
857   xiaomi    snapdragon  ...          1440           1
873   xiaomi    snapdragon  ...          2088           1
874   xiaomi    snapdragon  ...          1860           1
875   xiaomi    snapdragon  ...          1914           1
906   xiaomi          helio  ...           720           0

[980 rows x 24 columns]
```

## 25. Saving data in a new file

Code:

```
# 25 Saving data in a new file
df.to_csv("saved_file.csv", index = False)
```

Output:

|                                                                                                       |                  |                       |        |
|-------------------------------------------------------------------------------------------------------|------------------|-----------------------|--------|
|  LabExercises-Sem1 | 17-11-2025 16:39 | File folder           |        |
|  saved_file        | 17-11-2025 20:27 | Microsoft Excel Co... | 122 KB |

## 26. Using scipy basic functions

Code:

```
# 26 Using scipy basic functions
print(special.cbrt(27))
print(np.exp(3))
print(list(itertools.permutations([1,2,3])))
print(list(itertools.combinations([1,2,3],2)))
```

Output:

```
3.0
20.085536923187668
[(1, 2, 3), (1, 3, 2), (2, 1, 3), (2, 3, 1), (3, 1, 2), (3, 2, 1)]
[(1, 2), (1, 3), (2, 3)]
```

## 27 Using scipy – eigen values/vectors

Code:

```
# 27 Using scipy - eigen values/vectors
matrix = np.array([[2,1], [1,3]])
values, vectors = linalg.eig(matrix)
print("EigenValues: ", values, "\nEigen Vectors: ", vectors)
```

Ouput:

```
EigenValues: [1.38196601+0.j 3.61803399+0.j]
Eigen Vectors: [[-0.85065081 -0.52573111]
 [ 0.52573111 -0.85065081]]
```

## 28 Optimization of any function

Code:

```
# 28 Optimization of any function
func = lambda x: (x-5)**2
min_value = optimize.minimize(func, x0= 2)
print(min_value)
```

Output:

```
message: Optimization terminated successfully.
success: True
status: 0
  fun: 2.5388966380964196e-16
   x: [ 5.000e+00]
  nit: 2
  jac: [-1.697e-08]
hess_inv: [[ 5.000e-01]]
  nfev: 6
  njev: 3
PS: S:\MTech\DataScience\DataScienceLaboratory\datascience-learning>
```

## 29 Import dataset and check unique values and number of columns

Code:

```
#29 Import dataset & check unique values and number of columns
print(df.nunique())
print(len(df.columns))
```

Output:

```
brand_name          46
InsertedNewColumn   13
model              980
price              379
Average_Rating      30
5G_or_not           2
processor_brand     13
num_cores           3
processor_speed     35
Battery_Capacity    89
fast_charging_available 2
fast_charging       32
ram_capacity        9
internal_memory     8
screen_size        79
refresh_rate        6
num_rear_cameras    4
os                  3
primary_camera_rear 18
primary_camera_front 19
extended_memory_available 2
resolution_height   65
resolution_width    40
Available_On_Flipkart 2
dtype: int64
24
```

### 30. Sum and cleaning dataset using dropna

Code:

```
# 30 Sum and cleaning dataset using dropna
print("Sum of Average Rating: ", df['Average_Rating'].sum())
clean_df = df.dropna()
print(clean_df)
```

Output:

```
Sum of Average Rating: 6878.9
  brand_name InsertedNewColumn ... resolution_width Available_On_Flipkart
2      apple          bionic ...           1242             0
42     apple          bionic ...            750             0
46      asus    snapdragon ...           1080             1
47      asus    snapdragon ...           1080             1
48      asus    dimension ...           1080             1
..      ...          ... ...           ...             ...
975   xiaomi    snapdragon ...           1080             0
976   xiaomi    snapdragon ...           1080             0
977   xiaomi    snapdragon ...           1080             0
978      zte    snapdragon ...           1080             1
979      zte    snapdragon ...           1116             1

[678 rows x 24 columns]
```

31. Using: isnull(), sum(), any(), unique(), nunique(), replace(), fillna(), interpolate()

Code:

```
# 31 Using: isnull(), sum(), any(), unique(), nunique(), replace(), fillna(), interpolate()
print(df.isnull().sum())
print(df.isnull().any())
print("Unique ScreenSize", df["screen_size"].unique())
print("\n NUnique Processor Spped", df['processor_speed'].nunique())
df = df.replace(0, np.nan)
df = df.replace("", np.nan)
df = df.fillna(df.mean())
df = df.interpolate()
```

Ouput:

```

brand_name      0
InsertedNewColumn 20
model           0
price           0
Average_Rating  101
5G_or_not       0
processor_brand 20
num_cores       6
processor_speed  42
Battery_Capacity 11
fast_charging_available 0
fast_charging    211
ram_capacity     0
internal_memory  0
screen_size      0
refresh_rate     0
num_rear_cameras 0
os               14
primary_camera_rear 0
primary_camera_front 5
extended_memory_available 0
resolution_height 0
resolution_width 0
Available_On_Flipkart 0
dtype: int64

```

```

brand_name      False
InsertedNewColumn  True
model           False
price           False
Average_Rating  True
5G_or_not       False
processor_brand  True
num_cores       True
processor_speed  True
Battery_Capacity  True
fast_charging_available False
fast_charging    True
ram_capacity     False
ram_capacity     False
internal_memory  False
screen_size      False
refresh_rate     False
num_rear_cameras False
os               True
primary_camera_rear False
primary_camera_front True
extended_memory_available False
resolution_height False
resolution_width False
Available_On_Flipkart False
dtype: bool

```

```

Unique ScreenSize [6.1 6.5 5.4 6.7 5.42 6.06 6.71 6.68 6.73 4.7 6.78 6.8 5.9 4.
6.4 6.3 6.58 3.54 6.26 6.2 6. 5.7 6.34 6.14 6.6 6.57 6.67 6.76
6.75 6.81 5.84 6.53 6.74 8. 7.8 6.52 6.82 6.95 6.91 6.62 6.56 6.44
5. 6.09 5.45 5.5 6.92 6.54 6.55 6.28 6.47 6.9 6.43 6.51 6.59 6.49
6.22 7.1 7.4 5.86 6.72 7.2 5.3 6.83 7.6 6.66 6.38 6.39 8.03 7.
6.35 4.5 5.2 6.64 6.36 7.92 8.01 8.02 6.18]

```

```

NUnique Processor Sped 35
s:\MTEch\DataScience\DataScienceLaboratory\datascience-learning\LabExercises-Sem1\experiments\DataAnalysis\DataAnalysis.py:13
9: FutureWarning: DataFrame.interpolate with object dtype is deprecated and will raise in a future version. Call obj.infer_ob
jects(copy=False) before interpolating instead.
df = df.interpolate()

```



### 32 Maximum, average based on condition

Code:

```
# 32. Maximum, average based on condition
print(df[df["Battery_Capacity"] > 7.5]['Battery_Capacity'].max())
print(df[df["Battery_Capacity"] > 6.5]['Battery_Capacity'].mean())
```

Result:

```
22000.0
4817.748194014448
```

### 33. Basic Statistics

Code:

```
# 33. Basic Statistics
print(df.describe())
```

Output:

|       | price         | Average_Rating | 5G_or_not  | ... | resolution_height | resolution_width | Available_On_Flipkart |
|-------|---------------|----------------|------------|-----|-------------------|------------------|-----------------------|
| count | 980.000000    | 879.000000     | 980.000000 | ... | 980.000000        | 980.000000       | 980.000000            |
| mean  | 32520.504082  | 7.825825       | 0.560204   | ... | 2214.663265       | 1075.852041      | 0.560204              |
| std   | 39531.812669  | 0.740285       | 0.496616   | ... | 516.484254        | 290.164931       | 0.496616              |
| min   | 3499.000000   | 6.000000       | 0.000000   | ... | 480.000000        | 480.000000       | 0.000000              |
| 25%   | 12999.000000  | 7.400000       | 0.000000   | ... | 1612.000000       | 1080.000000      | 0.000000              |
| 50%   | 19994.500000  | 8.000000       | 1.000000   | ... | 2400.000000       | 1080.000000      | 1.000000              |
| 75%   | 35491.500000  | 8.400000       | 1.000000   | ... | 2408.000000       | 1080.000000      | 1.000000              |
| max   | 650000.000000 | 8.900000       | 1.000000   | ... | 3840.000000       | 2460.000000      | 1.000000              |

[8 rows x 19 columns]

### 34. Find count of null values for each field

Code:

```
# 34. Find count of Null values for each field
print(df.isnull().sum())
```

Output:

```
brand_name          0
InsertedNewColumn   20
model               0
price               0
Average_Rating      101
5G_or_not           0
processor_brand     20
num_cores           6
processor_speed      42
Battery_Capacity     11
fast_charging_available 0
fast_charging       211
ram_capacity         0
internal_memory      0
screen_size         0
refresh_rate        0
num_rear_cameras    0
os                  14
primary_camera_rear  0
primary_camera_front 5
extended_memory_available 0
resolution_height    0
resolution_width     0
Available_On_Flipkart 0
dtype: int64
```

35. Rows with null values for ANY column

Code:

```
# 35 Rows with null values for ANY column
print(df[df.isnull().any(axis=1)])
```

Output:

```

    brand_name InsertedNewColumn ... resolution_width Available_On_Flipkart
0      apple      bionic ...      828      0
1      apple      bionic ...      828      0
3      apple      bionic ...     1170      1
4      apple      bionic ...     1170      1
5      apple      bionic ...     1170      1
..      ...      ... ...      ...      ...
901    xiaomi      helio ...      720      0
902    xiaomi      helio ...      720      0
903    xiaomi      helio ...      720      0
906    xiaomi      helio ...      720      0
971    xiaomi    snapdragon ...     1080      0

[302 rows x 24 columns]

```

### 36. Fill the null values with Zero

Code:

```

# 36 Fill the null values with zero
df_zero = df.fillna(0) |
print(df_zero)

```

Output

```

    brand_name InsertedNewColumn ... resolution_width Available_On_Flipkart
0      apple      bionic ...      828      0
1      apple      bionic ...      828      0
2      apple      bionic ...     1242      0
3      apple      bionic ...     1170      1
4      apple      bionic ...     1170      1
..      ...      ... ...      ...      ...
975    xiaomi    snapdragon ...     1080      0
976    xiaomi    snapdragon ...     1080      0
977    xiaomi    snapdragon ...     1080      0
978      zte    snapdragon ...     1080      1
979      zte    snapdragon ...     1116      1

[980 rows x 24 columns]

```

### 37. Drop rows if ANY column has null value

Code:

```

# 37 Drop rows if ANY column has NULL value
df_dropnull = df.dropna()
print(df_dropnull)

```

Output:

|     | brand_name | InsertedNewColumn | ... | resolution_width | Available_On_Flipkart |
|-----|------------|-------------------|-----|------------------|-----------------------|
| 2   | apple      | bionic            | ... | 1242             | 0                     |
| 42  | apple      | bionic            | ... | 750              | 0                     |
| 46  | asus       | snapdragon        | ... | 1080             | 1                     |
| 47  | asus       | snapdragon        | ... | 1080             | 1                     |
| 48  | asus       | dimensity         | ... | 1080             | 1                     |
| ..  | ...        | ...               | ... | ...              | ...                   |
| 975 | xiaomi     | snapdragon        | ... | 1080             | 0                     |
| 976 | xiaomi     | snapdragon        | ... | 1080             | 0                     |
| 977 | xiaomi     | snapdragon        | ... | 1080             | 0                     |
| 978 | zte        | snapdragon        | ... | 1080             | 1                     |
| 979 | zte        | snapdragon        | ... | 1116             | 1                     |

[678 rows x 24 columns]

### 38. Confirm no null values are removed

Code:

```
# 38. Confirm no null values are removed
print("Confirm no null values",df_dropnull.isnull().sum())
```

Output:

```
qfLb6: Tufeq
vL9IJ9pJ6 Ou fJfbk9Lf      0
L620JnfJou mTqff          0
L620JnfJou MeT8Mf          0
6xf6uq6q w6wouL vL9IJ9pJ6 0
bLJw9L c9w6L9 frouf        0
bLJw9L c9w6L9 L69L         0
02                          0
unw L69L c9w6L92           0
L6fL62p L9f6               0
2cL66u 2J26                0
Juf6Lw9J w6wouL            0
L9w c9b9cJfL               0
f92f c9L8Juf               0
f92f c9L8Juf vL9IJ9pJ6     0
B9ff6L c9b9cJfL            0
bL0c6220L 2b66q            0
unw c0L62                  0
bL0c6220L pL9uq            0
2c ol uof                  0
vL6L986 89fJuf             0
bLJc6                       0
w0q6J                       0
Juf6L6qM6MCOJnuw          0
COufJw no unJJ v9Jn62 pL9uq v9w6 0
```

### 39. Melting, lambda functions, pivot

Code:

```
# 39. Melting, lambda functions, pivot
melted = pd.melt(df, id_vars=["price"])
print("Melted", melted)
pivoted = df.pivot_table(values='Average_Rating', index="ram_capacity", aggfunc='mean')
print("Pivoted Table: ", pivoted)

df['lambda_col'] = df["Average_Rating"].apply(lambda x: x* 10)
print(df['lambda_col'])
```

Output

```
Melted      price      variable  value
0      38999      brand_name  apple
1      46999      brand_name  apple
2     109900      brand_name  apple
3      51999      brand_name  apple
4      55999      brand_name  apple
...      ...      ...      ...
22535    13999  Available_On_Flipkart      0
22536    14439  Available_On_Flipkart      0
22537    16490  Available_On_Flipkart      0
22538    19999  Available_On_Flipkart      1
22539    61990  Available_On_Flipkart      1

[22540 rows x 3 columns]
```

```
Pivoted Table:      Average_Rating
ram_capacity
2              6.133333
3              6.373333
4              7.163679
6              7.873077
8              8.353943
12             8.620408
16             8.675000
18             8.750000
```

```
0      73.0
1      75.0
2      77.0
3      74.0
4      75.0
...
975    75.0
976    77.0
977    80.0
978    82.0
979    89.0
Name: lambda_col, Length: 980, dtype: float64
```